RAPPORT DE PROJET W2 : SYSTÈME DE SAUVEGARDE AUTOMATIQUE

Étudiants : HOMAWOO Anne-Marie, LOKOU Ayawa Gracia & AGBAVON essi Angele

Contexte : Administration Système sous Windows (VirtualBox)

---

1. INTRODUCTION

Dans le cadre de notre formation en administration système, nous avons été chargés de réaliser le **Projet W2**. L'objectif de ce projet est de mettre en place une véritable politique de backup (sauvegarde) automatisée.

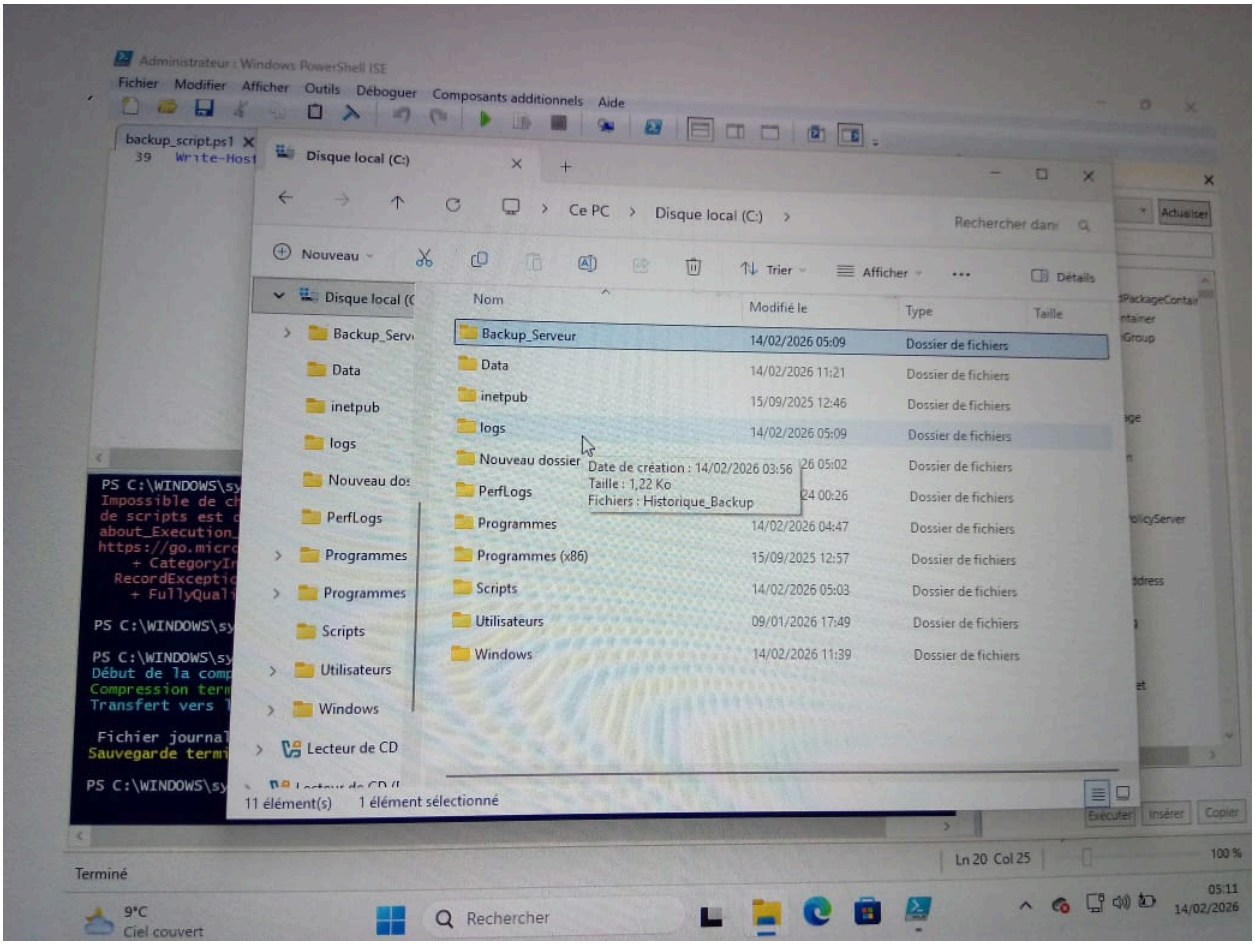
Pour répondre à ce besoin, nous avons conçu un script PowerShell capable de compresser un répertoire de données, de transférer cette archive vers un espace de stockage externe simulé, et de conserver une trace des opérations via un historique (logs). Enfin, nous avons automatisé ce processus pour qu'il s'exécute de manière autonome.

2. PRÉPARATION DE L'ENVIRONNEMENT ET DES DOSSIERS

Afin de simuler l'environnement d'une entreprise sur notre machine virtuelle, nous avons commencé par préparer notre arborescence de fichiers :

- **Le dossier source :** Nous avons créé un dossier `C:\Data` contenant divers fichiers (textes, documents) représentant les données sensibles à sauvegarder.
- **Le disque de destination :** Nous avons créé un dossier `C:\Backup_Serveur` à la racine de notre système pour simuler le disque dur externe ou le partage réseau.

- **Le dossier d'historique :** Nous avons mis en place un répertoire `C:\Logs` destiné à recevoir les rapports de nos sauvegardes.



3. DÉVELOPPEMENT DU SCRIPT POWERSHELL

Le cœur de notre projet repose sur un script PowerShell que nous avons nommé `backup_script.ps1`.

Notre démarche étape par étape :

1. Nous avons généré une variable de date dynamique pour que chaque archive ZIP ait un nom unique (ex: `Backup_2026-02-14_20-00.zip`).
2. Nous avons utilisé la commande `Compress-Archive` pour créer le fichier ZIP.
3. Nous avons utilisé l'outil `robocopy` pour déplacer l'archive vers la destination finale tout en générant un journal détaillé.

Voici le code de notre script :

PowerShell

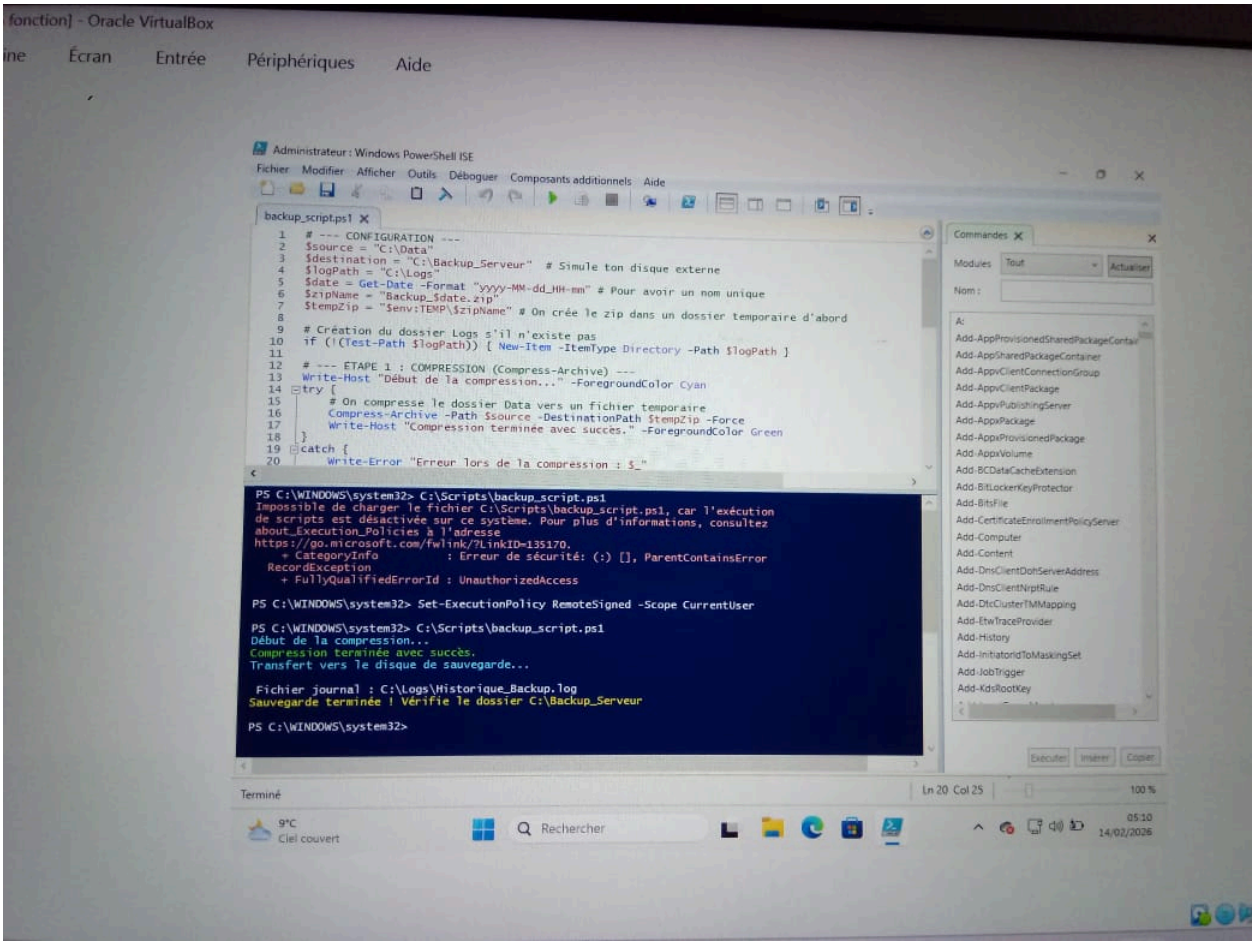
```
# --- CONFIGURATION DES CHEMINS ---
$source = "C:\Data"
$destination = "C:\Backup_Serveur"
$logPath = "C:\Logs"
$date = Get-Date -Format "yyyy-MM-dd_HH-mm"
$zipName = "Backup_$date.zip"
$tempZip = "$env:TEMP\$zipName"

# Création du dossier Logs si nécessaire
if (!(Test-Path $logPath)) { New-Item -ItemType Directory -Path $logPath }

try {
    Write-Host "1. Compression en cours..." -ForegroundColor Cyan
    # Compression du dossier Data
    Compress-Archive -Path $source -DestinationPath $tempZip -Force

    Write-Host "2. Transfert vers le serveur de Backup..." -ForegroundColor Cyan
    # Utilisation de Robocopy pour le déplacement et la gestion de l'historique
    robocopy $env:TEMP $destination $zipName /MOV /LOG+:"$logPath\Historique_Backup.log" /NP

    Write-Host "SAUVEGARDE TERMINÉE AVEC SUCCÈS" -ForegroundColor Green
}
catch {
    Write-Host "ERREUR : $($_.Exception.Message)" -ForegroundColor Red
}
```



4. AUTOMATISATION DE LA TÂCHE

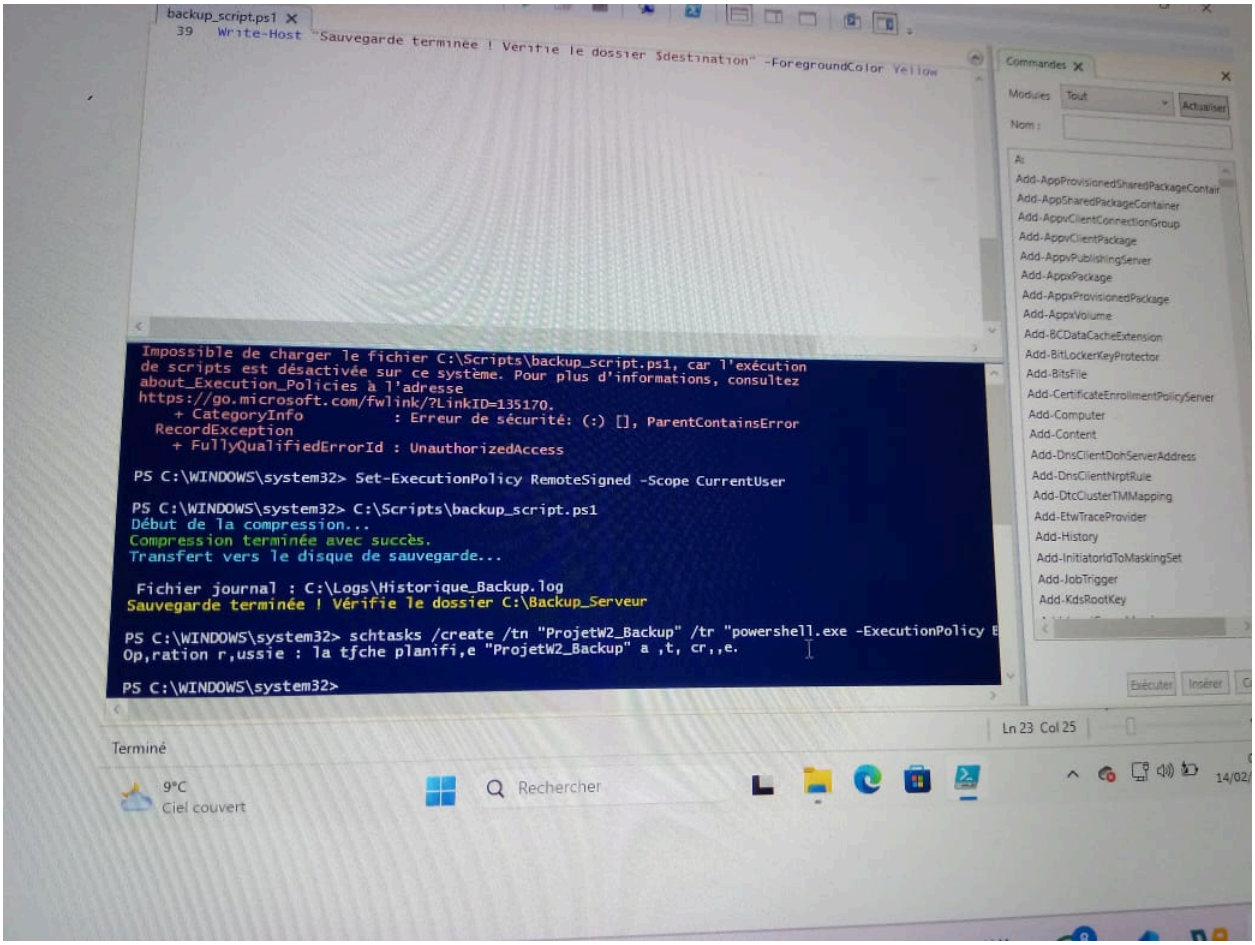
Pour répondre à l'exigence d'une planification automatique, nous avons utilisé le Planificateur de tâches de Windows via l'invite de commande (outil `schtasks`).

Nous avons configuré notre système pour que le script de sauvegarde s'exécute de façon transparente **tous les jours à 20h00**.

Commande exécutée :

DOS

```
schtasks /create /tn "ProjetW2_Backup" /tr "powershell.exe -ExecutionPolicy Bypass -File C:\Scripts\backup_script.ps1" /sc daily /st 20:00
```



5. RÔLE DES OUTILS ET COMMANDES UTILISÉS

Afin de mieux comprendre les mécanismes mis en place, voici un tableau récapitulant les commandes clés de notre projet et leur fonction concrète :

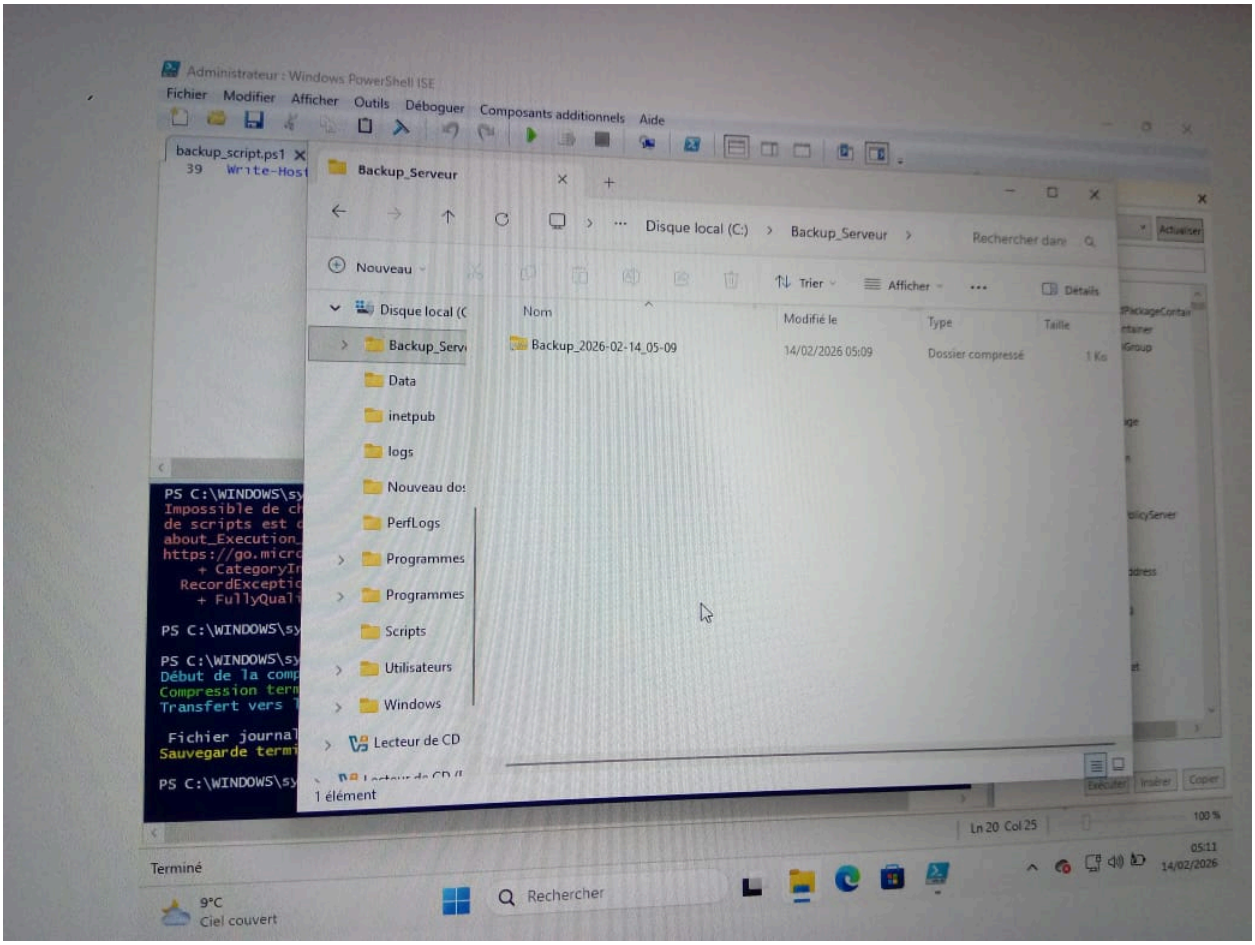
Commande / Outil	Rôle concret dans notre projet
Get-Date	Permet de récupérer l'heure système pour dater nos fichiers de sauvegarde et éviter qu'ils ne s'écrasent.
Compress-Archive	Outil natif PowerShell demandé dans le cahier des charges pour créer le fichier ZIP à partir de notre dossier source.
Robocopy	Utilitaire très robuste que nous avons utilisé avec l'argument `/LOG+` pour générer l'historique de la

	sauvegarde sans effacer les anciens logs, et `/MOV` pour déplacer le fichier final.
Schtasks	Commande Windows qui nous a permis de communiquer directement avec le Planificateur de tâches pour automatiser le lancement du script.
Set-ExecutionPolicy	Commande indispensable que nous avons utilisée en amont pour autoriser Windows à exécuter notre fichier `.ps1`.

6. CONCLUSION ET TEST DE RESTAURATION

Ce projet W2 nous a permis de mettre en pratique une véritable procédure de sécurité informatique.

Pour valider notre travail, nous avons procédé à un **test de restauration** : nous avons volontairement supprimé les fichiers du dossier source `C:\Data`, puis nous avons extrait le contenu de notre dernière archive présente dans `C:\Backup_Serveur`. Les données ont été entièrement restaurées. Le système est donc fiable, documenté via les logs, et totalement autonome.


Administrateur : Windows PowerShell ISE
Fichier Modifier Afficher Outils Déboguer Composants additionnels Aide
backup_script.ps1
Backup_Serveur
Disque local (C:) > Backup_Serveur
Nouveau
Trier
Afficher
Détails
Nom
Modifié le
Type
Taille
Backup_2026-02-14_05-09
14/02/2026 05:09
Dossier compressé
1 Ko
Data
inetpub
logs
Nouveau dos
PerfLogs
Programmes
Programmes
Scripts
Utilisateurs
Windows
Lecteur de CD
1 élément
Terminé
Ln 20 Col 25
9°C
Ciel couvert
Rechercher
05:11
14/02/2026